Miniproject: Natural Language Processing

Nathan Wong

Semester 1, 2025

Contents

Project overview and summary

The aim of this miniproject was to learn the basics of neural networks as applied to a simple problem in natural language processing: text classification. Textual ideas such as tokenisation, lemmatisation, and stop words was the first focus. Then the project moved onto converting textual data into numerical inputs via vectorization, with TF-IDF vectorization being used as inputs to the first couple of classification models. Early work involved the Python NLTK library before moving on to SKlearn.

Baseline machine learning models such as logistic regression and support vector machine were used as a first taste of classification algorithms. The fundamentals of deep learning were then introduced, with one week devoted to learning the basics of PyTorch. Throughout the rest of the project Keras would also be used as an easier, higher-level way to implement the variety of the deep learning models.

The bulk of the project consisted of implementing different neural networks for text classification and comparing their performance across different training data sizes, hyperparameters, and network depth. These networks included a one-dimensional CNN, an LSTM, an LSTM with pretrained word embeddings using both Word2Vec and GloVe, and then finally a pretrained BERT. The corpus used throughout was a set of Amazon reviews of various products, classified as either positive or negative reviews. The entire corpus being much too large to fit into Google Colab's runtime, selections of 20000, 50000, and even 100000 reviews were fed into the models, as long as the runtime didn't crash.

Throughout the project the goal was to increase the performance of the various models. The baseline models were around 85% accurate on validation test data, with initial feed-forward networks and CNNs underperforming in comparison. This was likely due to not enough training samples (limited by the computing environment), the requirement for simple networks (again limited by computing environment), and unoptimised hyperparameters. Additionally,

it was found that traditional neural networks are unable to take into account the text as a sequence of tokens, and as context plays a heavy role in the meaning of language this was a detriment to model performance. However, this part of the project nevertheless resulted in much exploration of the internal architecture of a CNN, including the central idea of a convolution, the importance of pooling and dropping layers, and the calculations required to determine the number of inputs in each successive layer given parameters such as the kernel size, stride length, and padding.

From there a simple LSTM was implemented with word embeddings (not pretrained). The internal workings of an LSTM were thoroughly investigated, with particular emphasis placed on learning how they are able to take into account the sequential nature of natural language through their processing of the data in time steps through LSTM cells. Initial performance was only marginally better than the CNN; but once pretrained embeddings were used, accuracy increased to just under 90%.

After using a pretrained transformer model for the text classification task, I devoted the final week to comparing the models for the same set of hyperparameters. This involved learning about accuracy metrics such as the F1 score and how a confusion matrix can be used to visualise model performance. In this part of the project I also learned about why the different models behave as they do and the strengths and weaknesses of each model.

What follows is a project log, in which I document not only what I did but also my understanding of various concepts as I learned them—including the basic concepts of natural language; how neural networks learn at a basic mathematical level; how Python libraries such as `torch`, `keras`, and `sklearn` can be used to implement models, the internal architecture and advantages or disadvantages of CNNs, LSTMs, and transformers; and the effect of hyperparameters on model training and performance. All code can be found here and the dataset for the text classification problem can be found here. (Many of the notebooks are not documented at all, because I was learning at the same time I was programming and any attempt to provide accurate documentation would have been futile because of how quickly the code was changing. The final two weeks' notebooks are actually documented, at least to an extent, because I finally felt that I was getting somewhere.)

Week 1: Introduction to NLP

Natural Language Processing (NLP) uses artificial intelligence to understand and generate text in human languages. NLP is already ubiquitous in everyday life, whether it be with voice-powered assistants and online chatbots. What's exciting about NLP is its potential to extend into the realm of sentiment analysis—that is, its ability to analyse and recognise emotions, feelings, sarcasm, mood, and other insights from largely unstructured text data. This, in turn, constitutes a remarkable leap in data analysis.

NLTK

Natural Language Toolkit (NLTK) is a Python library designed for processing of English text. Here are some key concepts related to text processing. - Tokenising: breaking up text into its words - Stop words: repeated words that don't add much to meaning and are usually featured out (e.g., 'in', 'as', 'of') - Stemming: reducing words to their root - Lemmatizing: reducing words to their core meaning (e.g., best -> good) - Chuncking: breaking up text into phrases—often the meaning of individual words doesn't bear much relation to the phrase that it's in

This week I began experimenting with the NLTK library, writing a simple script to tokenize and lemmatize a text file before printing out the most common words, both before and after taking out stop words. A plot of the frequency distribution is also generated and saved to a file.

Code

Week 2: Deep learning and PyTorch

Deep learning and neural networks

(Appended here a few weeks down the road, after having learnt more about the whole idea.)

Deep learning is a subfield of machine learning in which models learn via multiple layers of units called 'neurons', in a nod to the structure of the human brain. The model learns by mathematically tweaking a set of parameters called weights and biases that affect what output is produced by the model given a particular input. The set of layers of neurons and the weights and biases are called a neural network.

There are many different kinds of neural networks, but here is an overview of the basic idea.

The central concept is that of a neuron, which is just a cell holding a real number. A series of neurons makes up a layer of the network. In a traditional, fully-connected neural network, each neuron is connected to every neuron in the previous layer. These connections hold weights, and they provide a formula for how to obtain the value of this neuron from the neurons in the previous layer. This formula is just a weighted sum of the neurons in the previous layer, plus an extra term called the bias. In essence, the calculation being performed is a linear transformation of the previous layer—this makes it easy to do calculus down the road.

Let's get more concrete. Suppose in layer i the neurons are $x_1^i, x_2^i, \ldots, x_n^i$ with associated weights $w_1^i, w_2^i, \ldots, w_n^i$. Let the bias associated with neuron k in layer j be b_k^j. From the preceding discussion, the value of the neuron x_k^{i+1} is

$$x_k^{i+1} = \sum_{t=1}^{n} x_t^i w_t^i + b_k^{i+1}.$$

But if we only ever perform linear transformations on our data, non-linear relationships are going to be impossible for the model to learn. Therefore, we need to introduce some kind of non-linearity to the network. This is done by passing the weighted sum through an 'activation function'. Common activations functions include the sigmoid function,

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

and the ReLU function,

$$\text{ReLU}(z) = \max(0, z).$$

So we can refine our previous formula to be

$$x_k^{i+1} = f\left(\sum_{t=1}^{n} x_t^i w_t^i + b_k^{i+1}\right)$$

where f is some activation function of our choosing. Some activation functions will be better than others, depending on the context and the specific kind of neural network being trained.

We can package all the weights corresponding to the previous layer for the neuron x_k^{i+1} into a column matrix denoted by W_k^{i+1} and all the previous layer neurons into another column matrix denoted by X^i. Then the calculation becomes a simple dot product:

$$x_k^{i+1} = f\left(W_k^{i+1} \cdot X^i\right).$$

Even better, we can put the column vectors W_k^{i+1} for all k from 1 to n into a matrix W^{i+1} of dimension $n_i \times n_{i+1}$, where n_i and n_{i+1} are number of neurons in layers i and $i+1$, and the biases for each neuron in layer $i+1$ into a column matrix b^{i+1}. Then *every* neuron in layer $i+1$ can be calculated via simple matrix multiplication:

$$X^{i+1} = f\left({W^{i+1}}^T X^i + b^{i+1}\right).$$

The neurons in the first layer are the inputs to the network. For example, if the network was going to recognise images that were 32 by 32 pixels in size, the first layer would have 1024 neurons. The final layer depends on what we want the network to do with the input. For example, if we were classifying the images as either 'dog' or 'not a dog', the output layer might have a single neuron, with its value corresponding to the probability that the model thinks that image is a dog or not.

Gradient descent and backpropagation

Initially, the weights and biases of a network are random, and so its output is basically guaranteed to be nonsense. So how does the network learn?

The idea is to treat the network has a function of an enormous number of variables—one for each weight and bias. This is because the weights and biases are the only parameters that can be tweaked to make the network perform better. What we also need is a measure of how well the model is currently performing; this is provided by something called the 'cost function.'

The cost function is a function that takes in the output of the neural network as well as the expected output, and then computes a numerical value that indicates how far from the expected output the model's output is. A simple and commonly used loss function is the mean squared error, which is just the average of the square of the deviations of the prediction from the expected output. (We square the differences rather than taking the absolute value again because that is more friendly for calculus.) If Y is the vector for the expected output and $\hat{Y}$ is the prediction, the mean squared error can be calculated by the formula

$$\frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2.$$

What's important is that the smaller the cost function, the closer the prediction to the expected output. So we want to tweak the weights and biases to minimise the cost function. This is a classical optimisation problem that calculus is an excellent tool for solving.

Let's call the cost function C. Its inputs are the expected output and every single weight and bias of the model. The expected output is not really a variable because, given a particular input, it doesn't change and so we can consider C as a function of all the weights and biases only. We then compute the *gradient*, ∇C, of this multivariate function—that is, we compute a vector whose entries are the partial derivatives of C with respect to all the weights and biases. The gradient of a function of n variables gives the direction in Euclidean n space in which the function increases most rapidly; therefore the negative of the gradient gives the direction in which the function *decreases* most rapidly. So if W is a vector containing all the weights and biases of the model, we update W to get more favourable values (that is, lower cost) of the weights and biases by setting it equal to $W - \nabla C$. Rinse and repeat and over time, (we hope) the weights and biases will converge to the values that minimise the cost function and thus produce the desired output from a given

input. This mathematical idea is called gradient descent.

To actually compute the gradient of a function with so many variables, an algorithm called backpropagation is used. Conceptually, this means starting by calculating the partial derivatives with respect to the weights, biases, and activations in the final layer, and then using those values to calculate the partial derivatives of the previous layer, and so on.

Let's go back to the equation

$$x_k^{i+1} = f\left(\sum_{t=1}^{n_i} x_t^i w_t^i + b_k^{i+1}\right).$$

From the chain rule

$$\frac{\partial C}{\partial w_j^i} = \frac{\partial C}{\partial x_k^{i+1}} \frac{\partial x_k^{i+1}}{\partial w_j^i}$$

with $\frac{\partial C}{\partial x_k^{i+1}}$ already known from performing the algorithm on a previous layer and

$$\frac{\partial x_k^{i+1}}{\partial w_j^i} = f'\left(\sum_{t=1}^{n_i} x_t^i w_t^i + b_k^{i+1}\right) \times x_j^i.$$

Similarly

$$\frac{\partial C}{\partial b_k^{i+1}} = \frac{\partial C}{\partial x_k^{i+1}} \frac{\partial x_k^{i+1}}{\partial b_k^{i+1}}$$

and

$$\frac{\partial x_k^{i+1}}{\partial b_k^{i+1}} = f'\left(\sum_{t=1}^{n_i} x_t^i w_t^i + b_k^{i+1}\right) \times 1.$$

Although not needed for the gradient ∇C, we also compute, for each neuron m in the previous layer,

$$\frac{\partial C}{\partial x_m^i} = \sum_{k=1}^{n_{i+1}} \frac{\partial C}{\partial x_k^{i+1}} \frac{\partial x_k^{i+1}}{\partial x_m^i}$$

with

$$\frac{\partial x_k^{i+1}}{\partial x_m^i} = f'\left(\sum_{t=1}^{n_i} x_t^i w_t^i + b_k^{i+1}\right) \times w_m^i.$$

Note that there is a sum here because the neuron x_m^i appears in the calculation for every neuron in layer $i+1$, so using the chain rule requires us to sum over all the neurons in layer $i+1$.

By computing $\frac{\partial x_k^{i+1}}{\partial x_m^i}$ for each m in the previous layer, we can now compute the partial derivatives with respect to the weights and biases of the previous layer exactly as we have just done for this layer.

What happens when we start out, at the final layer? In that case we haven't already computed the partial derivatives with respect to the neurons in the layer in front, because there is no layer in front; but we can appeal directly to the derivative of the cost function because the final layer neurons are inputs in the cost function.

There's one final catch. Training the model doesn't rely on a single training example—often the examples come in batches. So the gradient for each example in a batch is computed, and then averaged to get the overall gradient for that particular batch. In symbols,

$$\nabla C = \frac{1}{n} \sum_{i=1}^{n} \nabla C_i$$

where n is the batch size and ∇C_i is the gradient computed from the output of the model on the ith input in the batch. Of course, the larger the batch size the closer the gradient to the true gradient; but the whole process becomes more computationally expensive. This idea, of calculating the gradient from batches, is called stochastic gradient descent.

PyTorch

The basics of using PyTorch to create a neural network is as follows.

- Import training and testing data using the `Dataset` class and wrap it up with `Dataloader` to allow it to be fed to the model in batches.
- Define a class that inherits from `nn.Module`; this class should have a `forward` function that passes the inputs through the model. This function can explicitly apply linear transformations to the data, or the class can be initialised with an instance of the `Sequential` class that makes it easy to define a sequential series of layers for the data to be fed through. Use `nn.linear` to create a module that applies a linear transformation to the input data.
- Choose a loss and parameter optimisation function.
- To train the model, enumerate the training data dataloader (which will come in batches of the batch size), calculate `cost` using the loss function and then use `cost.backward()` to calculate the gradient with respect to each parameter in the model. The `step` method of the optimiser then updates the parameters. Use the `zero_grad` method of the optimiser to reset the gradient—otherwise it accumulates.
- To test the model, use `torch.no_grad` to ensure no gradients are computed.

- Train and test the model over a number of epochs.

As a very simple example to get started, I generated values of the sine function at many inputs and implemented my own `Dataset` class to load the data from a CSV file. Then I trained a simple model to calculate the sine of its single input.

Code

(The model does well—but only on data within the range of its input! Passing 100π to the model, for example, returns rubbish.)

Week 3: Vectorisation and classification algorithms

To pass text to a classification algorithm or model, it must be vectorised. This is because an algorithm can operate on text. Algorithms for turning text into numerical input are called vectorisers.

A simple vectoriser is called Bag of Words, where each word in the text is associated with its frequency count in the text. While simple, this approach results in common words such as prepositions being weighted higher than other words that may be more important to the meaning of the document.

TF-IDF

A more complicated approach is called term frequency-inverse document frequency, or TF-IDF. This algorithm is applied to a collection of texts (corpus). The term frequency, TF, of a particular word in a particular document is f/n, where f is the number of times the word appears in the document and n is the number of words in the document. The inverse document frequency, or IDF, is $\log(N/F)$, where N is the number of documents and F is the number of documents in which the word appears. The TF-IDF number for that word is the product of its TF and IDF scores.

This means that words common to all the documents are given little weighting, which is appropriate because words like 'the' appear in virtually everything and don't add much meaning to a document. Words that appear frequently in one document but not the others will be given a higher weighting, and this is appropriate because it's much more likely that word is significant to the meaning of that document.

The TF-IDF vectorisers found in toolkits such as `sklearn` implement smoothing and other normalisation to change the TF-IDF scores slightly from the

simplistic model described above, but the idea is the same.

Logistic regression

Once a corpus has been vectorised, it can be fed into a classification algorithm. One such algorithm is called logistic regression, which is similar to linear regression except that the output is often binary (categorical), not continuous. This is useful, for example, in classifying whether an email is spam or not, or whether a movie review is positive or negative.

The idea is to take an input vector, say x, and then apply a linear transformation to it in the form $z = w \cdot x + b$, where $\cdot$ represents the dot product. However, this z could be any real number, and therefore we then pass it to the function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

which spits out a number between 0 and 1. This gives us a probability that the input belongs to one of the two specified classes. We can then apply a decision boundary to determine whether the probability is high enough for us to conclude that the input belongs to that category—for example if $\sigma(z) > 0.5$ we conclude that the input vector belongs to the positive class.

What we want is the output $\sigma(z)$ to represent the probability that x belongs to the positive class. Denote by y the class that the input vector belongs to—that is, either 0 or 1. So in symbols we have

$$P(y = 1) = \sigma(z).$$

Since the probabilities must add to 1, we also have

$$P(y = 0) = 1 - \sigma(z) = \sigma(-z).$$

Now we require an appropriate loss function. Remember that we require $P(y) = z$ when $y = 1$ and $P(y) = 1 - z$ when $y = 0$; an easy way to capture this is

$$P(y) = z^y(1 - z)^{1-y}.$$

We would like the parameters in our model to maximise this probability. We take the logarithm of both sides:

$$\log P = y \log z + (1 - y) \log(1 - z);$$

maximising this is the same as minimising

$$-\log P = -y \log z - (1-y)\log(1-z)$$

and this function is our loss function, called binary cross entropy loss. For multiple predictions, we simply take the average of the losses for each individual prediction.

The weights and biases that go into the linear transformation of the input are learnt by stochastic gradient descent.

Support Vector Machine

Another classification algorithm is called Support Vector Machine (SVM). In this method inputs are represented as points in n-dimensional space, and the idea is to find the hyperplane that best separates the two groups of data. This is done by maximising the distance between the hyperplane to its closest points.

Mathematically, a hyperplane in Euclidean space is given by

$$w^T x + b = 0,$$

where, as is the usually the case, w is referred to as the weights and b the bias. If the input data is linearly separable—that is, there exists a hyperplane that cleanly separates the data into two classes, the SVM algorithm tries to find two hyperplanes that separate the two classes such that the distance between the two planes, the 'margin', is as large as possible. For example, one class can be determined by all points above the plane

$$w^T x + b = 1$$

and the other all points below the plane

$$w^T x + b = -1.$$

The distance between the two planes is $2/||w||$, and so the problem becomes that of minimising $||w||$ or $||w||^2$. However, this optimisation problem is subject to the constraint that no points fall into the margin. That is, if y_i, the label of the ith input, is 1, for example, then we require

$$w^T x_i + b \geq 1,$$

and similarly if $y_i = -1$. This is summarised by the constraint

$$y_i(w^T x_i + b) \geq 1,$$

thus turning into the problem into a multivariate optimisation problem with a constraint—which can possibly be solved using the method of Lagrange Multipliers.

When the data is not linearly separable, things become messier. One way to get around this is to introduce a "soft margin", where some data points are allowed in the margin and the goal is still to maximise the distance between the planes but taking into account these bad points.

Project update

First file is some basic experimentation with a TF-IDF pipeline in NLTK. But for actual use in a classification problem, I used `TFIDFVectorizer` in `sklearn` instead, which is a bit more advanced than the home grown method.

TF-IDF Pipeline using NLTK

SVM and Logistic Regression using sklearn

This week I used both of these classifiers on some reviews on Amazon. The corpus was split into two categories: positive and negative reviews. Where the models' predictions were different to the actual label, I have saved those reviews to separate files for analysis.

SVM Model Results

Logistic Regression Model Results

Some of the predictions seem to be plain wrong; but others seem to be wrong when there is some subtlety or nuance to the review's overall opinion. For example, in one review, a book was praised as an overall 'good read' but most of it was spent criticising the style of the author.

Week 4: Neural networks for text classification

This week I moved on from baseline models to neural networks.

Convolutional Neural Networks have the same basic structure as other neural networks: the input, a series of hidden layers in which transformations are made to the data via weights and biases, and then the output layer.

A convolutional neural network is a special type of neural network in which some of the layers are convolutional layers. This means instead of connecting every neuron in the previous layer to every neuron in the next (fully connected layer), the next layer is obtained by sliding a 'window' called the kernel or filter over the input layer; this window consists of weights and biases that are applied to the part of the input over which the kernel covers. Thus the next layer consists of the outputs of these convolutions. The idea is that different filters can recognise different features of the input. The CNN is built up of several convolutional layers, each with multiple filters, each recognising different aspects of the input.

To avoid overfitting, a technique called pooling is used, in which the size of the output of a convolutional layer is reduced by taking the average or maximum of a group of outputs, increasing computational efficiency in the process. A CNN often ends with fully connected layers, which take the output of the previous convolutional and pooling layers to produce the final output of the network.

CNNs are often used for image recognition–two dimensional data. In this case I have built a CNN for my one dimensional input data, which is just text. The text is vectorised using TF-IDF, and the input to the network consists of a tensor of TF-IDF values. The final output layer of the CNN has only one neuron, as the task is classifying whether the text is positive or negative: the closer to 1, the higher the chance of the text being positive.

CNN text classifier

With only a single convolutional and fully connected layer, the validation test results reach about the same level of accuracy as the baseline model (around 86%). Increasing the number of epochs or the number or layers only resulted in the validation accuracy decreasing due to overfitting. To increase the accuracy of the model, it is likely more data is needed.

(Strangely, the model gets stuck using PyTorch but not Keras... Even with PyTorch as the backend for Keras.)

Project update

The PyTorch network doesn't seem to be learning at all—the cost is always around 0.7 or thereabouts, and no changes to any of the hyperparameters seems to affect it. This is true with both the CNN and the simple fully connected network. However, when the network is implemented using Keras and run on the same input data (TF-IDF vectorisation of the dataset), the model does seem to learn and the validation accuracy reaches around 85% (roughly on par with the baseline classifiers). And using PyTorch as the Keras backend gives the same result...

A head scratcher, for now. CNN text classifier with Amazon reviews

After fiddling around with the learning rate, the manual implementation seems to work with a learning rate of 10^{-3}. Mysteriously, sometimes the cost won't go anywhere and remains stuck at around 0.7, other times it will jump around but decrease overall, resulting in validation test accuracy of around 83%.

Week 5: RNNs and LSTMs

Normal neural networks assume that each piece of data in the input is independent of the other pieces of data. Unfortunately, this assumption is not correct for textual data, because often the meaning of a word or phrase depends on the words that have come before it. To a human this idea of context is intuitive; but traditional neural networks are unable to capture this link.

Recurrent Neural Networks (RNNs) are a kind of neural network that aims to solve this problem. As such, they are often used on streams of textual data to predict the next word at the end of a phrase or sentence.

RNNs work by keeping a 'memory' of the input data that has been received up until a particular time; RNNs work in time steps, with each time step involving the processing of one piece of the input (e.g., the next word). The processing of each piece of input is done by 'cells.' The idea behind a RNN is that each cell accepts the next piece of input and produces an output called the 'hidden state' that is then fed into the next cell, taking into account its memory of what has occurred in the previous time steps. As with CNNs, the output of the recurrent layers is fed into a dense layer that spits out the output required by the network.

How these cells actually work relates to the particular type of RNN being used. One type of RNN is called Long Short-Term Memory (LSTM) and is the network I tried to implement this week for my text classification problem.

Embeddings

One thing not done in the previous weeks' foray into text classification using NNs was the inclusion of an embedding layer. In this case it was necessary because the input to the LSTM needs to be sequential. In earlier weeks the input consisted of sparse arrays of vectorised texts (using TF-IDF), where each

word always occupied the same index in the array. However, this completely disregards the sequential aspect of text processing, as well as the relationship that might arise between words in the context of a particular review. Word embeddings are a way to solve this problem.

First I used a vectoriser to create a dictionary of unique words in the corpus, assigning to each word an integer or index. Then I converted each piece of text into an array of indices to represent the words, padding them to be the same length as the longest review. This could be passed directly into a network, but no success was achieved. :(

Then I found out about embeddings, which are a way of converting those arrays of indices into higher dimensional vectors, whose entries are initialised randomly. As the network learns, the entries in those vectors are fine-tuned (learned) to reflect more accurately the semantic relationships between the words in the text. So essentially the embedding layer is just another layer in the network, which takes as input a list of indices (or this can be thought of as a large matrix of zeroes with a single 1 indicating which word is represented) and applied a linear transformation to it via the embedding matrix to output a vector for each word in the text. This output is then fed into the LSTM layer.

How LSTMs work

An LSTM layer consists of a series of LSTM cells, each of which process each piece of input data for for each time step. As input is fed through the LSTM, something called the cell state is maintained, which acts like the memory of the LSTM. Each cell produces an output called the hidden state, which is then fed into the next cell as part of its input.

So each LSTM cell takes three pieces of input: h_{t-1}, the previous hidden state, c_{t-1}, the previous cell state, and x_t, the current input. These three pieces of data are fed through three 'gates':

- forget gate: this decides which information in the cell state is relevant (and ''forgets'' the rest);
- input gate: this decides which information in the input (x_t) is relevant and then uses it to produce a new cell state;
- output gate: this determines the hidden state to be output from the cell, given the new cell state.

The forget gate

To ease notation, denote by $L(x)$ a linear transformation of the vector x with a set of weights and a bias. Pointwise multiplication is denoted by $\otimes$ and pointwise addition is denoted by $\oplus$.

The computation for this gate is

$$f_t = \sigma(L(x_t) + L(h_{t-1})).$$

The values in f_i are between 0 and 1, and so multiplying this with c_t pointwise effectively ''forgets'' some of its values and ''remembers'' others.

The input gate

We now need to decide how to update the cell state. The change to the cell state is given by

$$\Delta c_t = \tanh(L(h_{t-1}) + L(x_t)).$$

In other words, we compute the change to the cell state via a linear transformation of the two inputs, H_{t-1} and x_t; but we also want to make sure that the irrelevant parts of the input are forgotten. This is achieved by computing

$$i_t = \sigma(L(h_{t-1}) + L(x_t));$$

that is, another sigmoid activate linear layer, but with a different set of weights and biases to the forget gate. Note that tanh is used because the output is between -1 and 1; this is because we might want to decrease some of the values in the cell state as well as add them.

Thus, to compute the new cell state, we first forget some of the old cell state, decide which parts of Δc_t we want to keep, and then pointwise add the two vectors together, viz.

$$c_t = c_{t-1} \otimes f_t \oplus \Delta c_t \otimes i_t.$$

The output gate

We want to now output some part of the cell state—but only the part most relevant to the input and previous hidden state. So we compute, again,

$$o_t = \sigma(L(h_{t-1}) + L(x_t))$$

and then spit out the new hidden state

$$h_t = o_t \otimes \tanh(c_t).$$

(Here tanh is used to normalise the hidden states.)

And that's it! This new hidden state is then fed into the next LSTM cell with the next input, and this along with the current cell state goes through all the same operations again.

Project update

Finally got the homebaked LSTM to work—accuracy approximately 82%. I needed to flatten the output of the LSTM differently so that all time steps was passed to the dense layer; with this fix somehow the model started to actually train.

Text classifier using LSTM in PyTorch and Keras

Week 6: Pretrained embeddings

This week was about using pretrained word embeddings.

Word2Vec

One method of generating word embeddings is Word2Vec. This is a simple 2-layer neural network, with the input being sparse one-hot encoded arrays (or perhaps indices) and the output being either a target word or its context. The two main ways in which the embeddings are developed are called Continuous Bag-of-Words (CBOW) and Skipgram. The former predicts a target word based on the surrounding sentence as input, whereas the latter predicts the context based on the target word. The weights of this network are the word embeddings. The 'window' in the training method is the number of words on either side of the target/input word to be taken as context with either the CBOW or the Skipgram method.

I grabbed Google's massive pretrained Word2Vec embeddings and loaded them into my LSTM model using `nn.Embedding.from_pretrained`. I also collected validation test data for the different epochs to make a plot. With the pretrained embeddings and setting the `freeze` parameter to `False` (allowing the pretrained embeddings to be updated during the training of the model), the validation test accuracy crept up to around 89%. With the same set of samples and values for the hyperparameters the vanilla LSTM model from last week only got up to around 86%.

TODO: try larger number of samples (if it doesn't crash Google Colab) and pretrained embeddings using other methods (e.g., GloVe).

Update: somehow 100000 samples is no longer too much for the runtime. However, the results for this were similar—maximum accuracy remained around 89% for the same set of hyperparameters.

LSTM with pretrained Word2Vec Embedding

GloVe

GloVe (Global Vectors) is an extension to the Word2Vec method. In addition to the usual algorithm, a large co-occurrence matrix is maintained, in which the entries count how many times each word appears next to every other word in the corpus (or in the context window for window sizes greater than unity). If we let this co-occurrence count be X_{ij} for words i and j, then the algorithm tries to minimise the function

$$J = \sum_{i,j} f(X_{ij})(w_i \cdot w_j + b_i + b_j - \log(X_{ij}))^2,$$

where w_i and w_j are the embedding vectors (weights) for the words and f is a weighting function designed to reduce the impact of frequent words that don't add much semantic meaning.

LSTM with pretrained GloVe Embedding

Week 7: Transformers and BERT

The next level up from LSTMs with pretrained embeddings is transformers. Transformers are a deep learning architecture that have revolutionised the field since becoming established in a famous 2017 paper titled 'Attention is All You Need'. Transformers evolve on RNNs via a mechanism known as attention, which allows it to deduce dependencies over a much larger range than even LSTMs. Moreover, the architecture of a transformer is well suited to parallel computation, allowing them to take full advantage of GPUs and therefore also the advantages of massive amounts of training data.

Transformer architecture

Transformers accept tokens as embeddings, with an additional positional encoding added to each embedding vector to encode where that token belongs in the context of the entire input text.

These embedding vectors then pass through a series of attention layers and feedforward neural networks. The attention layers are where the model learns about the relationship between different tokens in the input text.

In a single head of attention, each embedding vector is transformed by a matrix of weights, W_Q, that produces a 'query' vector. This vector can be thought of as a piece of information that the model is searching for. Similarly, there is another matrix of weights, W_K, that produce a series of 'key' vectors, which are like the answers to those queries. If a key vector matches a query vector, then that token is relevant to the query. The actual meaning of the queries is quite obtuse, because the model learns to query the information it needs to for the cost of its prediction to decrease; but it might be something like whether there is a new proper noun nearby that might modify a token representing a relative pronoun.

If we denote by Q the matrix of all query vectors and K the matrix of all key vectors, the matrix $Q^T K$ gives a matrix in which the entries correspond to how strongly queries are answered by particular keys. For some obscure reason, in the original paper, this matrix is scaled down by $\sqrt{d_k}$, the dimension of the query/key space (a user-supplied parameter).

Since the numbers in this attention matrix can be arbitrarily large, they are then passed column wise into the softmax function, which turns each column into a probability distribution. This distribution can be thought of as expressing how likely a particular key is to answer a particular query. These probabilities can also be thought of as weights. The softmax function is given by

$$\text{softmax}(z) = \frac{e^z}{\sum e^{z_i}}.$$

What to do with these weights? These weights express how likely a particular key is to answering a particular query. A third matrix of weights, called the value matrix and denoted by W_V, is then multiplied by every embedding to create a series of value vectors. These vectors can be thought of as describing what needs to be added on to a particular embedding vector to move that vector in a direction in the embedding space that aligns more closely with the token associated with the key. The weights calculated in the attention matrix decide which parts of each value vector will contribute to a change in a particular embedding, so we simply have to multiply the attention matrix by the matrix whose columns are value vectors. This means the output of a single head of attention, given query, key, and value matrices Q, K, and V respectively, is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q^T K}{\sqrt{d_k}}\right) V$$

where d_k is the number of dimensions in the query/key space. The columns of this output are then added to the embedding vectors to update them, and that's a single head of attention complete! (Remember that the matrices Q, K, V are obtained by multiplying the embeddings by the corresponding matrices of tunable weights W_Q, W_K, W_V.)

Each attention block in a transformer then has multiple heads of attention running in parallel, each with different query, key, and value matrices. And this is where transformers harness the parallel computing power of GPUs. The reason for multiple attention heads is for the model to learn how context affects the meaning of the tokens in as many different ways as possible.

After the attention block the embeddings are fed into a feedforward network that consists of two linear transformations with a ReLU activation layer between them. The combination of the self-attention block and feedforward block is the basic unit of a transformer and repeated many times. More explicitly, a general transformer consists of a stack of encoder units, which have the structure just described, along with a set of decoder units, which are similar except they also have an encoder-decoder attention block. This means that the queries come from the previous decoder layer, but the keys and values come from the output of the encoder. The idea is that the encoder learns about the various connections between the tokens in the input text as those embedding vectors are being transformed, while the decoder takes those transformed embeddings to generate the required output text (this was the original use proposed for transformers—translating text from one language to another).

BERT

BERT, or Bidirectional Encoder Representations from Transformers, is a kind of transformer that includes only the encoder, not a decoder. The word 'bidirectional' is in the name because transformers allow all the text in a given input to be processed at once, and hence in any direction (left to right or right to left), whereas traditional RNNs can process input only in a specified order. The goal of BERT is to produce a transformer model able to learn relationships between words and sentences that can then be fine-tuned to a particular NLP task. For example, here I am fine-tuning BERT to turn it into a binary text classifier.

Project update

The main difficulty here was the amount of memory and computing resources required; Google Colab couldn't handle 50000 samples in the corpus at the same time as the transformer model, so I reduced it down to 20000. Even on the GPU, training for five epochs would have taken more than an hour or more; but even with the reduced training size and only a single epoch, the model reached around 92–96% accuracy on the validation data—a big improvement on the LSTM, CNN, and baseline models. It is likely that with the same amount of training data and the same number of epochs, the accuracy would increase even further...

Classifier based on fine-tuned BERT

Week 8: Hyperparameter tuning and model comparison

Project experiments

Hyperparameters are parameters such as the batch size, learning rate, number of layer, the dropout rate for networks such as CNNs, and the activation function used between the hidden layers and on the output layer. Varying these parameters influences how well the model performs; but the choices are also constrained by the computing resources available.

Throughout this project, a number of different hyperparameter configurations were tried during often frustrating periods of the models not learning. This was particularly apparent with the CNN and LSTM models in PyTorch. In both cases, it was eventually the learning rate that did the trick. The learning rate determines how much the model's weight change in response to the cost of its output compared to the expected output. A familiar analogy is of a hiker walking down a hill in three dimensional space. The goal, of course, is to find the bottom of the hill (the minimum of the cost function). A large learning rate means that the hiker takes very large steps, sometimes overshooting the optimal location or oscillating between it; a low learning rate means that the hiker takes very small, careful steps in the direction of steepest descent down the hill, but because of the small steps it takes longer to reach the destination.

With a default learning rate of 10^{-3}, the original CNN model's cost function fluctuated around 0.69 without any real improvement. Changing the learning rate to 10^{-2} didn't improve the model's performance. It only made the fluctuations in the cost more erratic. Eventually a learning rate of 5×10^{-4} resulted in the model actually learning (although it must be admitted that I tinkered around so much with the model out of frustration here I can't be entirely sure what caused it to start working—especially since the Keras

model was working just fine with the default learning rate).

The complexity of the model also has to be considered before training. For example, in a CNN, how many filters are there going to be? How many convolutional layers before the feedforward layer or layers? Adding more filters and output channels to the convolutional layers, or increasing the number of convolutional layers, results in a more complex network; but the end result is that the model starts to experience overfitting after fewer epochs, with not much of a chance in the maximum accuracy achieved on the testing data. Of course, the complexity of the model is also determined by the complexity of the data. With my vectorised corpus consisting of a single sequence of real numbers as input, a single convolutional and pooling layer was enough for the model to achieve around 83%.

With the LSTM, parameters included the number of LSTMs stacked on top of each other (layer depth) as well as the size of the hidden layer in each LSTM cell. Once the model started working (this time it didn't work because I needed to flatten the output of the LSTM so that all the information from the previous time steps was being fed into the linear network), increasing the hidden size from 20 to 100 didn't result in much improvement in the training accuracy (but the running time definitely increased!). Similarly, using pretrained word embeddings increased the accuracy, but once the pretrained embeddings were there, increasing the layer depth or size of the hidden state didn't have as big of an impact.

Another hyperparameter that could be varied is the number of epochs. A higher number of epochs is not better, because the model will be limited by the training data it has available. With the amount of data I had, the accuracy of the base LSTM model seemed to plateau after around 5-10 epochs, with overfitting very clear after 10 epochs. Interestingly, once the pretrained embeddings were increased, the testing accuracy peaked at a much earlier epoch, and then steadily decreased, suggesting overfitting is happening at a much earlier time. Plotting the testing accuracy versus the epoch allowed me to see how the model performed over time and the optimal number of epochs.

Hyperparameter tuning strategies

In this project I have tuned hyperparameters manually. However, there are a number of ways that this can be done more efficiently and automatically.

A naive method is called grid search, in which all possible value of a hyperparameter is established and then models constructed for all those possible

values before the best model is selected. This is, of course, inefficient.

A slightly better method is random search, where instead of going through all possible values of the hyperparameter, specific values are selected at random for each iteration. This is less resource-intensive than grid search.

A more advanced method is called Bayesian optimisation, in which the problem is treated like a mathematical optimisation problem. Here a probabilistic model is built that predicts the performance of the original model given a particular value of the hyperparameter. The probabilistic model is then improved after each iteration before selecting a likely value that will improve model performance for the next iteration.

There are also tools that automate hyperparameter tuning, such as Ray Tune, Keras Tuner, and Google's Vertex AI.

Performance metrics

The simplest metric of how well a model performs is its accuracy, the number of samples that it classifies correctly divided by the total number of samples in the training dataset. However, this metric can be unreliable when the dataset is not class balanced—that is, there are the same number of samples in each class. For example, if 90% of the dataset was of a particular class, the model could predict the same class for every input and end up with an accuracy of 90%.

Therefore a more nuanced metric is desirable. One of those is the model's F1 score. The two central ideas here are that of precision and recall. In a binary classification task, there are four possibilities for the output of the model:

- True positive (TP)
- True negative (TN)
- False positive (FP)
- False negative (FN)

Precision refers to what proportion of the outputs classified as positive are actually positive. The total number of inputs classified as positive is the sum of TP and FP, and so

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

Low precision means that the model predicts a lot of false positives.

Recall refers to what proportion of the actually positive inputs were classified as positive. The total number of actually positive inputs is the sum of TP

and FN, and so

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Low recall means that the model predicts a lot of false negatives.

The F1 score is the harmonic mean of precision and recall, which turns out to be

$$\frac{\text{TP}}{\text{TP} + (\text{FP} + \text{FN})/2}.$$

Using the harmonic mean means that the F1 score will improve the more similar recall and precision are. In a perfect world, we would want both recall and precision to be 1—so that there are no false positive or false negatives. A low F1 score indicates that there is a trade-off between precision and recall. For example, precision may be high, meaning that the model is likely to label positive inputs correctly, but at the cost of low recall, meaning that if faced with a negative input the model is still likely to categorise it as positive.

The notions of recall and precision, and hence the F1 score, are useful because they allow different perspectives on a model's performance in different scenarios. For example, in a text classification problem involving spam or genuine emails, users would be highly annoyed if a large number of real emails were blocked (low recall) even if the model does a good job of filtering out almost all spam emails (high precision). In this case high recall would be prioritised, even if precision suffers as a result.

A simple way to visualise the performance of binary classifier is a confusion matrix, which is 2 by 2 grid of numbers with the downward sloping diagonal being the number of true positives and true negatives and the other two slots being false positives and false negatives.

Model comparison

Summary and comparision of all models

BERT model

(Although the BERT model was implemented in the first file, somehow the runtime kept running out of memory; so I had to adapt the code from week 7.)

The two baseline models (logistic regression and SVM), the CNN, the LSTM, and the LSTM with pretrained embeddings (word2vec and GloVe) were all trained on a 30000 sample set, with 24000 samples dedicated to training and

6000 samples dedicated to testing. The batch size was 50 and the learning rate was 5×10^{-4}. All of those classifiers were trained over 10 epochs, with the F1 scores and confusion matrices computed from the epoch with the maximum accuracy (since different models began experiencing overfitting at different epochs).

Due to constraints with computing resources, the transformer was trained over only a single epoch with 20000 samples—16000 samples devoted to trainig and 4000 samples for testing. Additionally, after experimentation it turns out that the transformer requires a smaller learning rate of 10^{-5} to train. However, as can be seen from the table below, even with fewer training samples and epochs, the transformer still comfortably outperformed all the other models.

Accuracy scores, F1 scores, and confusion matrices of the results follow.

Table 1: Accuracy and F1 scores

Classifier	Accuracy	F1-Score
Logistic regression	85.73	86.22
SVM	86.25	86.69
CNN	85.25	85.56
LSTM	82.00	82.50
LSTM with pretrained word2vec	86.80	87.17
LSTM with pretrained GloVe	87.33	87.72
BERT	92.30	92.09

Advantages and disadvantages

The baseline models are limited by their strictly mathematical machinery. When applied to a text classification problem, their algorithms cannot take into account the nuances of context—particularly the order of the text—that are so important in extracting meaning from language. However, they are also much easier to implement and run without exorbitant computing resources.

An advantage of CNNs over fully connected networks is their ability to distinguish particular features of the input with different filters, with the pooling layers also reducing computational complexity and the chances of overfitting. However, if the input to the CNN is a basic vectorised form of the text (such as TF-IDF), again the order of the tokens, as well as the

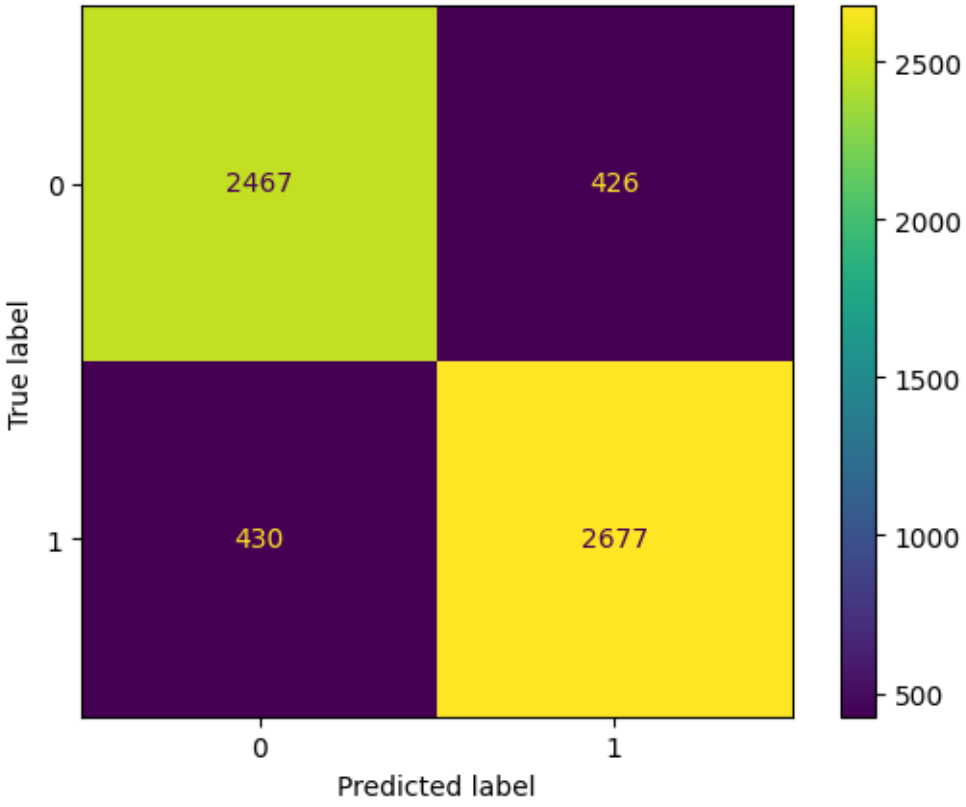

Figure 1: Logistic regression confusion matrix

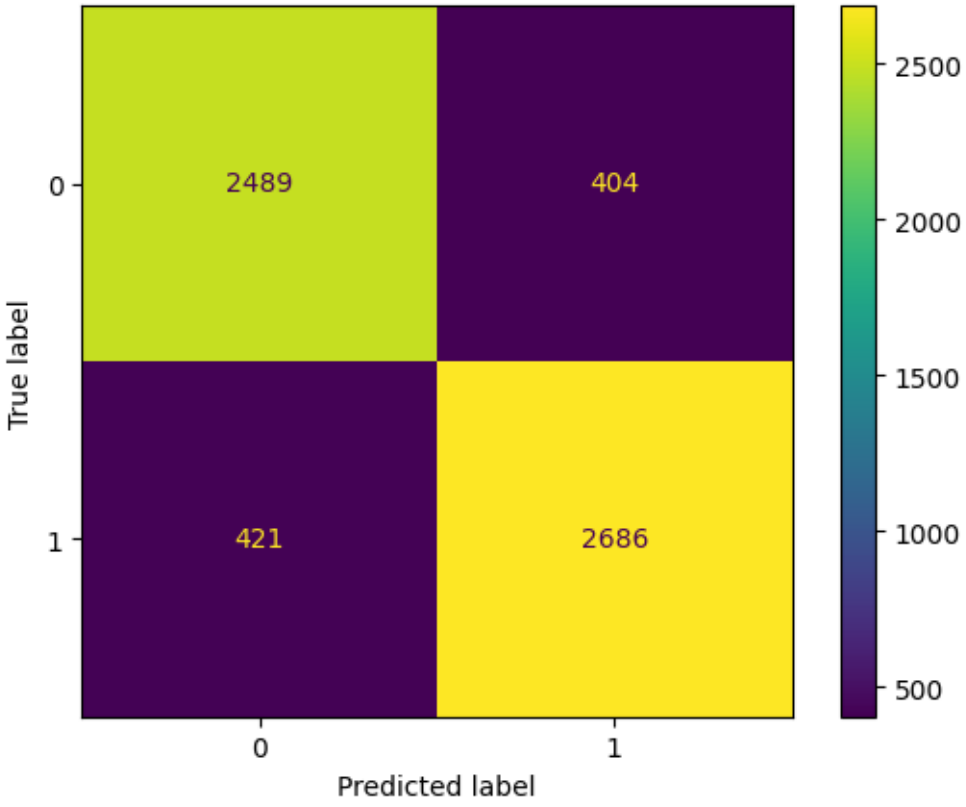

Figure 2: SVM confusion matrix

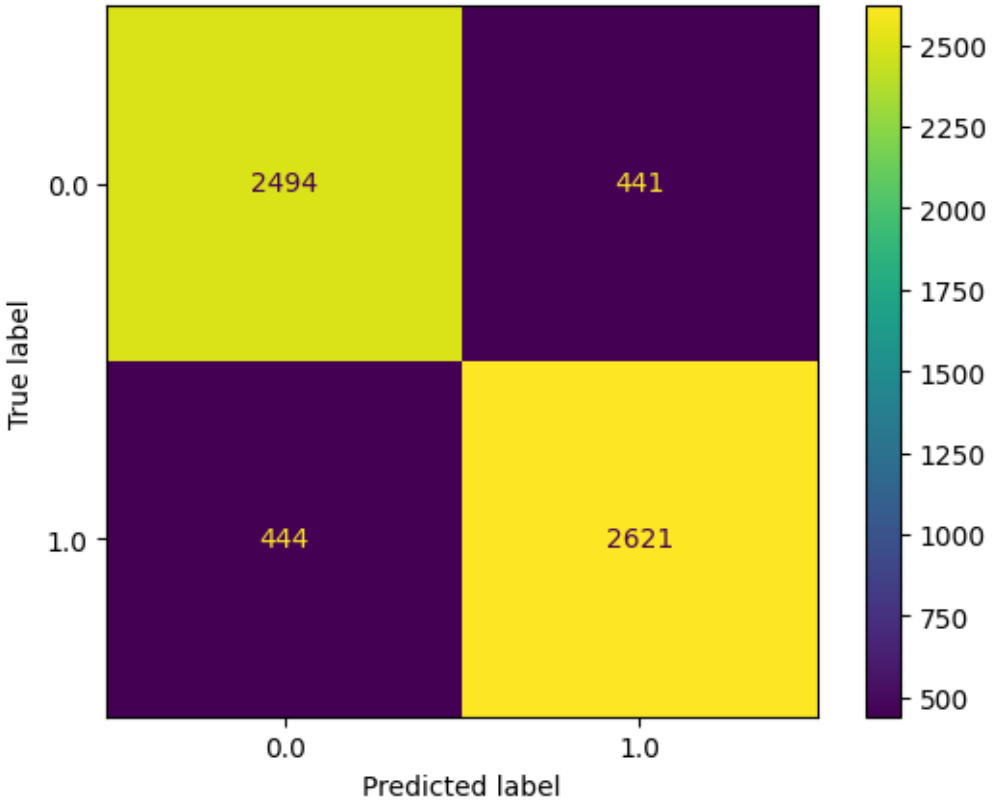

Figure 3: CNN confusion matrix

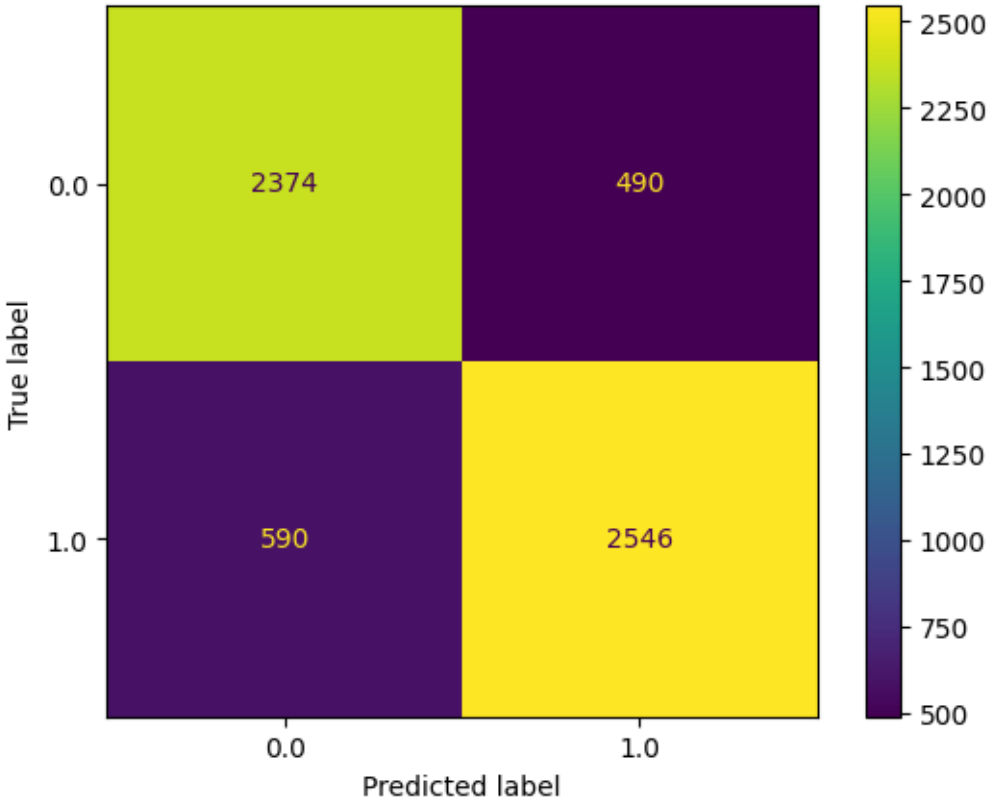

Figure 4: LSTM confusion matrix

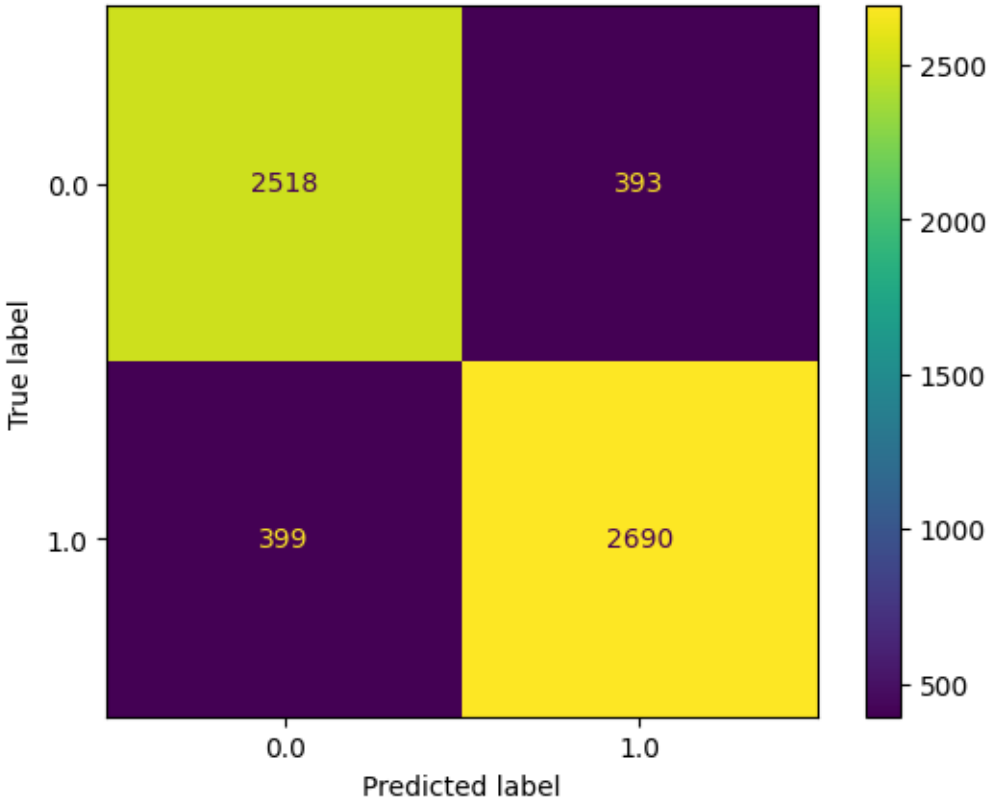

Figure 5: LSTM with word2vec confusion matrix

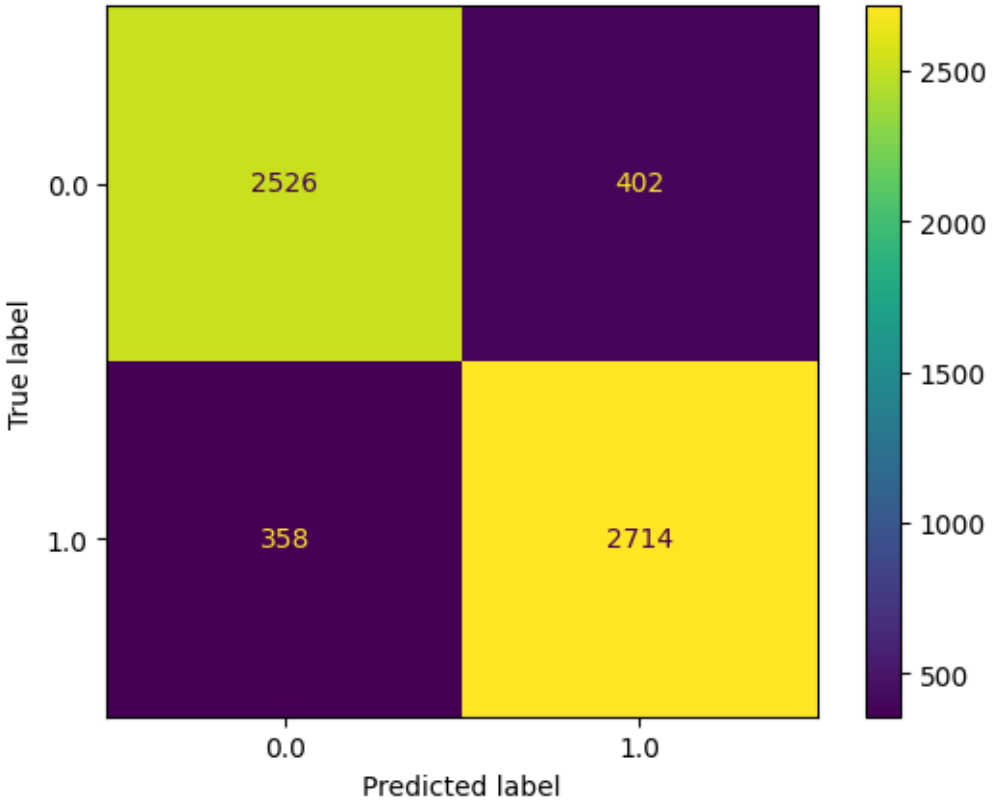

Figure 6: LSTM with GloVe confusion matrix

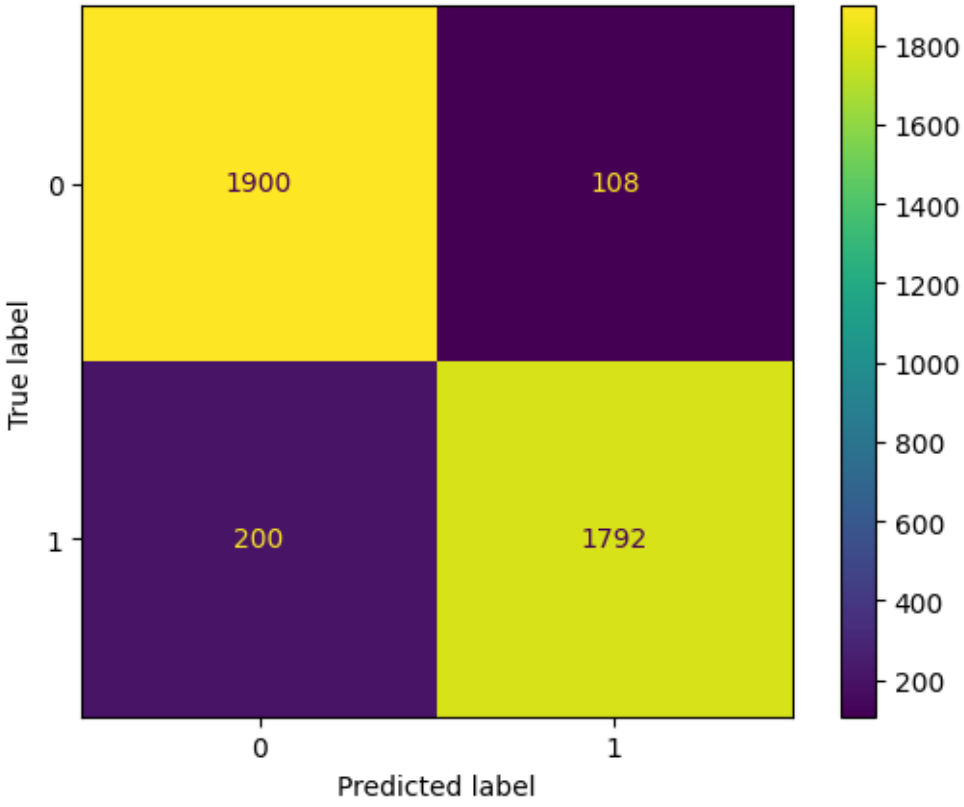

Figure 7: BERT confusion matrix

relationships between the words in a particular sample, cannot be considered by the network.

For this reasons RNNs and LSTMs quickly took over for the processing of sequential input. Since they process the input in time steps, LSTMs are able to take into account the order of the tokens. Additionally, their mechanism of hidden states and cell states encoding both the short and long-term memory of the network allows it to handle textual dependencies in a way that traditional feedforward networks cannot. The main limitation of LSTMs, which led to the development of attention and then transformers, is that their understanding of context arises in only dimensions, namely from the start of the sequence to the end; and in conjunction with the problem of vanishing gradient, this limits its ability to pick up on longer range textual dependencies. In addition, the sequential nature of the network can be a bottleneck for computation. However, when combined with pretrained word embeddings that are then fine-tuned by the model, LSTMs offer an improvement in performance over CNNs and the baseline models, as can be seen from the results.

Transformers take the trophy because their self-attention mechanism is able to pick up on the relationships between words in any direction and over a much larger range. Furthermore, multi-head attention is naturally suited to parallel computation by GPUs, making them able to perform many more computations at once and thus more effectively glean the complex nuances of the input text. Unfortunately, without a GPU (or when one runs out of time on the GPU on Google Colab), a transformer is not particularly practical when it comes to the time it takes to train even a simple model.

Challenges and future directions

The biggest challenge in this project was getting the hang of both the theoretical side of deep learning, all the different kinds of neural networks, and NLP in general while also learning how to actually train those models in Python. Being already familiar with Python in general, I found the practical little easier, but it still required much head-scratching, consultation of documentation for `torch`, `keras`, `sklearn`, `NLTK`, and `gensim`, and more than occasional bouts of frustration when a model wasn't learning properly or there was some obscure runtime error with one of the libraries.

It was also more than a little frustrating that time on Google Colab's GPU is restricted, meaning that for some of the more resource intensive models I had to time their training so that they could be run on the GPU. Still, the restriction made me think more about balancing model complexity and the size of the input dataset with the performance of that model once it was trained, which was also a useful impetus to experiment with hyperparameters.

Improvements I could've made include documenting most of the Jupyter Notebooks a little more carefully, as well as commenting on various obtuse aspects of the code (including randomly commented out lines that I put in during debugging). With the model themselves, although I trained them many times and got slightly different performances each time, I documented only one of those instances for each model in the final report, due in part to computing resource constraints and also in part to me not looking ahead to the summary and comparison part of the project at earlier stages. Having a variety of different accuracy and F1 scores for the models would have painted a much clearer picture of how the models stack up against each other, although I'm happy it looks like the later, more advanced models outperform the earlier, simpler models.

Avenues for further investigation include a more systematic analysis of how the hyperparameters affect the model performance. In this project I just varied the hyperparameters manually, eventually settling on a set of parameters I could

use for model comparison; a more thorough, and perhaps automated, approach would perhaps reveal more insight into the what the optimal parameters might be and how they affect model performance in general. Increasing the number of samples in the training data would also make model comparison easier, since the more data there is the likelier the model performance will be limited by the intrinsic nature of the model and not the lack of training samples. Indeed, as the number of samples increased to 100000 the LSTMs with pretrained embeddings, in particular, performed better than when the number of samples was only 20000 or 30000. Unfortunately due to memory constraints such a large number of training samples could not be tested with the CNN or the transformer, and so a baseline of 30000 samples had to be used across all the models for comparison.

Apart from varying parameters such as the batch size, number of epochs, or model complexity, we could look also at varying the optimiser used. In this project the Adam optimiser was used across all models without much of a deep dive into its inner workings and how it is different to stochastic gradient descent (as the focus was more on language processing and text classification). Learning more about different kinds of optimisers and which tasks they are best suited to should be fruitful.

On the NLP side, the obvious extension to this project is multi-label classification, and this in turn could lead to more nuanced tasks such as sentiment analysis. The work on transformers also naturally leads beyond simple classification tasks to the realm of text generation, and clearly that is an exciting and rapidly-developing area of NLP in which the techniques used are state of the art. From my brief dive into transformers, I understand there are many variants on the basic transformer architecture, and this could also prove a worthy focus of future work.